

SAMSUNG PRISM · GENERATIVE AI HACKATHON
3RD EDITION 2026-27

Sentry

*A troubleshooting engine that
verifies before it answers.*

Theme ID: Theme 02 · Smart Guided Troubleshooting Engine

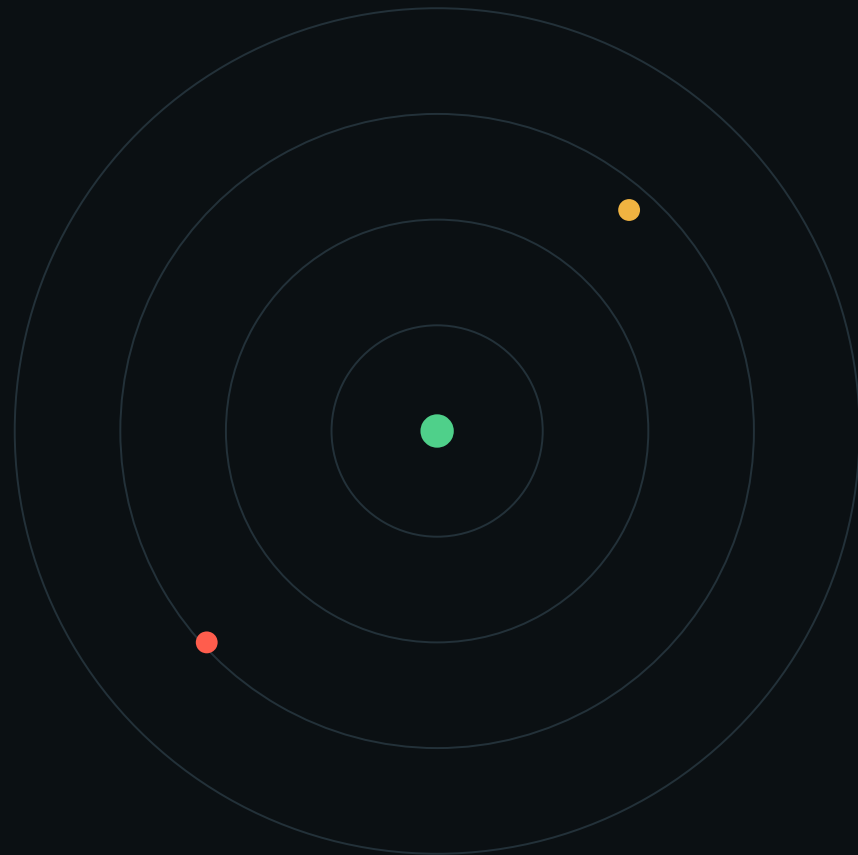
Team Ember · VIT Vellore

Manya Singh · manya.praveensingh2024@vitstudent.ac.in

Krishita Gupta · krishita.anandgupta2024@vitstudent.ac.in

Muskaan Arora · muskaan.arora2024@vitstudent.ac.in

[GitHub: https://github.com/manya-singh7/Sentry](https://github.com/manya-singh7/Sentry)



Smart Guided Troubleshooting Engine

Theme 02 · Vague device complaints in, validated one-tap troubleshooting plans out.

Input

WHAT COMES IN

A free-text complaint, plus optional SIIS reference text that the plan must be built from.

Output

WHAT GOES OUT

Pure JSON: goal, ordered actions, UI steps, catalog deeplinks, 8–10 query variations.

Speed

THEME TARGETS

P95 $\leq$ 300 ms for issues seen before. P95 $\leq$ 8 s for brand-new ones.

“Screen flickers and the battery dies fast.”

That's how customers really talk. Today an agent reads SIIIS knowledge-base articles, works out the cause and picks the steps by hand: about 15 minutes per scenario.

Then the customer still digs through nested Settings menus to apply the fix.

Two problems, one sentence, zero structure. That's the input we have to handle.

MANUAL TRIAGE · PER SCENARIO

~15 min

INTERACTIONS · GLOBALLY

Millions

Source: Samsung PRISM Theme 2 problem statement

Why not search, or a chatbot?

	GOOGLE	SENTRY
You give it	Keywords you have to guess	Your complaint, typed, spoken or photographed
It gives you	Ten links to read and compare	Ranked causes with a confidence score
The last mile	You dig through Settings yourself	One tap straight to the exact screen
Trust	You judge every source	Every link checked against the real catalog
When unsure	Always returns something	Asks one question, or says no match

Today's baseline is manual: an agent reads SIIS articles, about 15 minutes per scenario. Generic LLM chatbots are faster but invent help URLs and steps, which Theme 2 forbids. Sentry makes the decision, verifies it, and hands you the fix one tap away.

Complaint in. Fix out.

One API call turns a messy complaint into a ranked, validated, one-tap fix deeplinked straight to the right Settings screen.



Multimodal input

Text, voice, photo, or photo + sentence



Verified cache

Paraphrases served from cache, no LLM call



Self-critique

Catches and surfaces its own bad guesses



Honest fallback

Asks, or admits no match, instead of guessing



Describe a device fault by text, voice, or photo. Sentry checks it against a live troubleshooting catalog and returns a ranked repair plan.

■ Safe settings change ■ Needs your hands ■ Restart or reset

Try: Swipe inverted Battery drain Camera crash Dirty charging port Screen flickering

Describe the problem, for example: screen flickers when brightness is low. You can attach a photo too.

 Add photo

 Speak

Diagnose

sen·try /*'sen-trē/ noun*

A guard posted to watch, check whoever approaches, and stop what doesn't belong.



Watches

SELF-CRITIQUE

Rescores its own hypotheses against the complaint and shows you what it threw out.



Verifies

DEEPLINK VALIDATION

Every link is matched against the real 578-entry catalog. It never invents a URI.

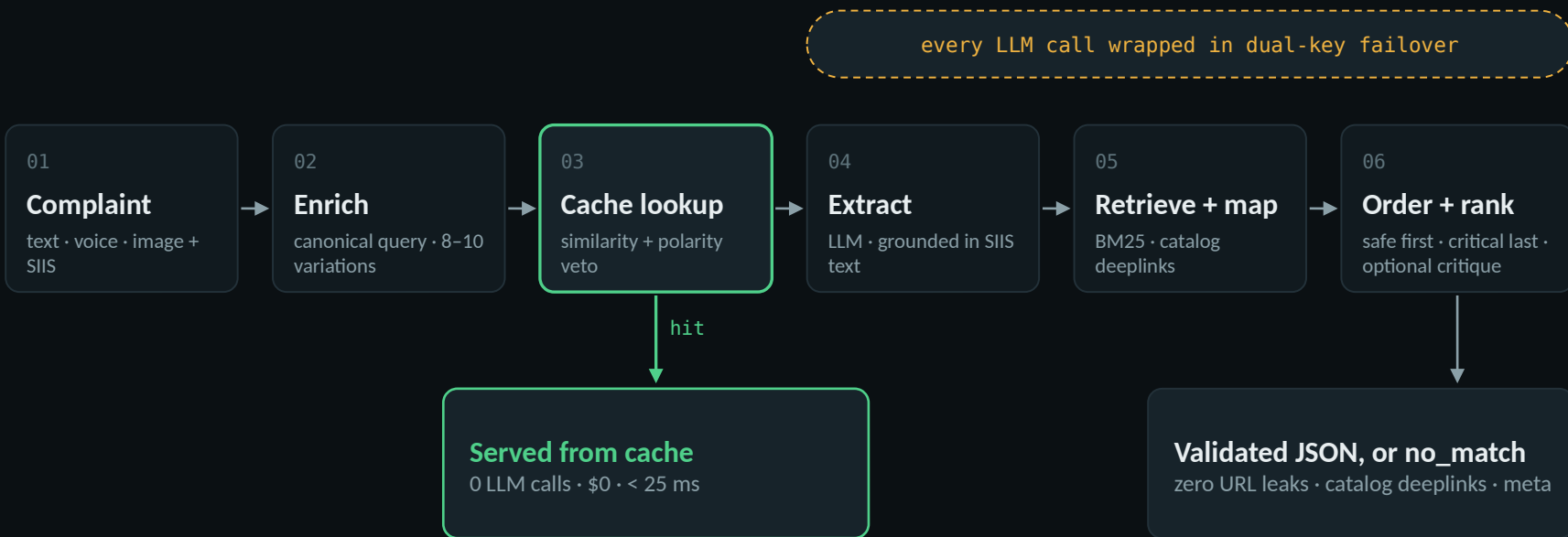


Stops

OFF-DOMAIN REFUSAL

Not a device problem? It says so, enforced by the prompt and a keyword backstop.

Six stages. One guarded pipeline.





Describe a device fault by text, voice, or photo. Sentry checks it against a live troubleshooting catalog and returns a ranked repair plan.

■ Safe settings change ■ Needs your hands ■ Restart or reset

Try: Swipe inverted Battery drain Camera crash Dirty charging port Screen flickering

my battery drains fast overnight

Add photo Speak Diagnose

Time **13 ms** Cache **Hit** Served from cache, matched "my battery drains fast overnight" (100%)
Model **gemini-3.6-flash** Cost **\$0.000000** Matched on this device Needs one more detail

Cache matches stay on the device: no API call, no cost, no data sent.

One question to narrow it down

Did this issue start with battery usage optimization, or does it involve screen brightness adjust?

Battery usage optimization Screen brightness adjust Both, or I'm not sure

< 25 ms

cache lookup, P95 on unseen paraphrases

\$0

per cached answer, no API call

≤ 300 ms

Theme 2 cache-hit target (P95)

"turn Bluetooth on" ≠ "off"

A polarity + entity veto stops near-identical embeddings from serving the wrong fix.

It shows you how sure it is.

One bold design choice: a radial gauge per hypothesis, instead of a buried number. Colour tells you at a glance whether to tap or to double-check.

● ≥ 0.90	50% land on the exact screen
● $0.70-0.89$	25% exact, rest fall back safely
● $0.50-0.69$	routes to a safe parent screen

Sample data, $n = 3 / 4 / 1$ goals per band; directional only. "Exact" = a specific catalog screen. Self-critique is optional (off by default).



Hypothesis 1 of 2: Battery

Battery usage optimization

Follow these steps to perform this Battery Fast Drain Troubleshooting

Verified via catalog retrieval, self-critique, and link validation, not a raw model guess.

► Why this diagnosis?

Configure Battery Saver Settings

Safe settings change

It will restrict background activity to extend overall battery charge life overnight.

1. Open Settings.
2. Tap Battery.
3. Tap Battery saver.
4. Turn on Use Battery saver.

[Open in Settings](#) voiceassist://dummy_positive

Optimize Background App Usage

Safe settings change

It will put unused apps to sleep and clean device system memory.

1. Open Settings.
2. Tap Device care.
3. Tap Optimize now.

[Open in Settings](#) voiceassist://dummy_positive

Reboot the Device

Restart or reset

It will restart device systems to clear temporary background process battery drains.

1. Press and hold the Power button.
2. Tap Restart.

It catches itself, out loud.

HOW IT'S VERIFIED

Automated tests (`test_self_critique.py`, 7/7 passing), including a mocked rejection of a weak hypothesis.

Not shown live: a live rejection only happens when the model produces a genuinely weak hypothesis.

Off by default. Enabled with `ENABLE_SELF_CRITIQUE=true`.

01

Rescore

A second pass checks each hypothesis against the original complaint.

02

Reject visibly

Discarded guesses are shown to the user, not quietly hidden.

03

Stay in budget

Runs only if under 4 s so far, with a hard 2 s timeout.

Describe it however it's easiest.



Describe a device fault by text, voice, or photo. Sentry checks it against a live troubleshooting catalog and returns a ranked repair plan.

■ Safe settings change ■ Needs your hands ■ Restart or reset

Try: Swipe inverted Battery drain Camera crash Dirty charging port Screen flickering

my phone battery dies out

Add photo Speak Diagnose

Got it. Review the text, then select Diagnose.



Describe a device fault by text, voice, or photo. Sentry checks it against a live troubleshooting catalog and returns a ranked repair plan.

■ Safe settings change ■ Needs your hands ■ Restart or reset

Try: Swipe inverted Battery drain Camera crash Dirty charging port Screen flickering

my phone battery dies out

Add photo Listening Diagnose

Listening. Speak into your microphone.



Describe a device fault by text, voice, or photo. Sentry checks it against a live troubleshooting catalog and returns a ranked repair plan.

■ Safe settings change ■ Needs your hands ■ Restart or reset

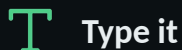
Try: Swipe inverted Battery drain Camera crash Dirty charging port Screen flickering

my phone is broken

demo.jpg (671.9 KB) Remove

Add photo Speak Diagnose

Sentry will read the photo and your description together.
Photo attached.



Type it

“phone gets hot while charging”



Say it

Voice, transcribed in the browser, then enriched



Show it

A photo of the problem, with or without a sentence

Tools & tech stack

FastAPI + Uvicorn

Pydantic

Google Gemini · gemini-3.6-flash

BM25 · rank_bm25

fastembed · all-MiniLM-L6-v2

Docker

Web Speech API

HTML / JS demo UI

Service

POST /v1/troubleshoot · GET /health · pure-JSON responses with latency, cache-hit and cost metadata.

Semantic cache is in-memory on the server. Dense embeddings are used for cache matching; deeplink retrieval is BM25.



From 15 minutes to seconds.

Use cases: agent assist in support centres, and self-service straight from the device's Settings or support app.

~15 min

manual triage per scenario today

5.04 s

cold-path P95 for a brand-new complaint

< 25 ms

repeat issues from cache, \$0 per answer

Sources: Theme 2 problem statement (15 min); team latency benchmarks. Industry context: TELUS × Google Cloud report 87% faster resolution and \$53.9M in annual savings with Gemini-based customer care (cloud.google.com/customers/telus-gemini-enterprise-cx).

Measured against the brief.

METRIC	THEME 2 TARGET	OURS
Schema conformance	100%	100% (sample set)
Web-URL leaks	0	0
Deeplinks valid in catalog	100%	100% (9/9, sample set)
Exact-screen resolution	Exact screen, not parent menu	69/73 offline mapping · end-to-end: MISSING
Step accuracy (0-3)	vs. reference samples	MISSING – needs to be measured
Cache-hit latency, P95	≤ 300 ms	< 25 ms (unseen paraphrases)
Paraphrase cache hit rate	≥ 80%	MISSING – needs to be measured
Cold-path latency, P95	≤ 8 s	5.04 s
Cost per query	Tracked	\$0.00027 cold · \$0 cache hit

SAMPLE SET = TEAM SAMPLE QUERIES (SMALL N). MISSING ROWS WILL BE MEASURED ON THE OFFICIAL DATASET.

We ran the ablation. Retrieval wins.

VARIANT	SCHEMA	CATALOG-VALID	P95	COST / QUERY
A · BM25 + label match (ours)	100%	100% (9/9)	5.0 s	\$0.00027
B · LLM only	70%	56% (5/9)	6.3 s	\$0.00084
C · Rules only	100%	78% (7/9)	5.0 s	\$0.00018

LLM-only invented non-catalog URIs on 3 queries and attached deeplinks to reboot steps.
Rules-only mapped “app” and “camera” to the wrong screens.

Ablation on the sample query set (9 actions). Step accuracy per variant: MISSING — needs to be measured.

CATALOG · ACTION → SCREEN

100% held-out · 18/18

94% development · 31/33

91% calibration · 20/22

69/73 correct, offline. 1 wrong screen; the other misses fell back to a safe screen.

Every failure has a ceiling.

COLD PATH, P95 vs. 8 s SPEC CEILING

5.04 s

8.0 s

WHAT BREAKS	WHAT SENTRY DOES	BOUND
Primary key hits quota (429)	Backup key takes over inside the same request	invisible
Both keys exhausted	Fails fast, no backoff; clean honest response	< 500 ms
Upstream 503 / timeout	One retry, then a clean fallback	≤ 8 s
Self-critique runs slow	Skipped past 4 s elapsed; hard 2 s timeout	≤ 8 s

What we'd tell you before you asked.

Late dataset

The official PRISM dataset arrived mid-hackathon. We built on sample data, then benchmarked against the real catalog once it landed.

Free-tier Gemini

Low per-minute and per-day request limits. The dual-key failover exists because of this, and it's tested, not theoretical.

One action per screen

Theme 2 requires one action per screen. We check it in code, but today a violation raises a warning instead of rejecting the plan. Hard enforcement is next.

Also open: small test sets; step accuracy, paraphrase hit rate and voice/photo accuracy not yet measured. None of these need an architecture change.

Yes.

*Can it become a worklet?
The path is short, not
theoretical.*

01 Swap in the full catalog

Retrieval was built on the same schema from day one. The official 578-entry catalog already runs with zero code changes.

02 Move the cache on-device

Today the cache runs in our backend: a hit makes no LLM call. Running the same lookup on the phone would keep repeat complaints private, in line with Galaxy AI.

03 Harden retrieval for real phrasing

Production users are messier than any hackathon set. The architecture absorbs more data without a redesign.

Beyond the brief.

Six things we built that Theme 2 didn't ask for.

01 Self-critique

Optional second pass rescores hypotheses and shows rejections. Verified by a mocked test.

02 Cache veto

Polarity + entity rules stop “Bluetooth on” getting the “off” fix.

03 Dual-key failover

A quota hit on one key switches to the backup inside the same request.

04 Multimodal input

Text, voice, photo, or a photo plus a sentence.

05 Clarifying question

When two causes are too close to call, it asks one question.

06 Off-domain refusal

Non-device complaints get no_match, not invented steps.

Theme 2 checklist.

REQUIREMENT	HOW	STATUS
Zero web-URL leaks	Programmatic scrub on every generated step	✓ Done
Catalog-only deeplinks	Matched on catalog descriptions; every URI validated	✓ Done
dummy_positive for unindexed screens	Used only when no catalog entry exists	✓ Done
No hallucinated steps	No viable fix → contexts: [] + fallback: no_match	✓ Done
Pure JSON delivery	Pydantic-validated, no markdown wrapping	✓ Done
Action order	Safe settings first, critical / destructive last	✓ Done
8–10 query variations	Generated at enrichment (ENABLE_QUERY_VARIATIONS)	✓ Implemented
One Action = One Screen	Checked in code; raises a warning, not a rejection	Partial
REST API	POST /v1/troubleshoot · GET /health	✓ Done
Containerised	Dockerfile with health check	✓ Done
Latency targets (P95)	≤ 300 ms cache hit · ≤ 8 s cold → < 25 ms · 5.04 s	✓ Met
Hit rate · step & screen accuracy	See slide 14	MISSING

Thank you.

Team Ember

Manya Singh · Krishita Gupta · Muskaan Arora · VIT Vellore

<https://github.com/manya-singh7/Sentry>

[Demo & project files \(Google Drive\)](#)

